Alarm Clock Monitoring System

MINI PROJECT REPORT

Engineering Research - Embedded Systems

EEE3305

Group Members:

1. J.M.Widanegamage-21UG0623
2. K.P.G.H.D.Dissanayake-21UG0618

Faculty of Engineering

Sri Lanka technological Campus

January 2024

Table of Contents

- Chapter 01 : Introduction
- Chapter 02 : Literature survey
- Chapter 03 : Methodology (Practical & Simulation design)
- Chapter 04 : Results and Discussion
- Chapter 05 : Conclusion and Future Work (including challenges)
- Appendices – Program Code
- References

CHAPTER 01 : INTRODUCTION

Alarm Clock Monitoring using PIC Microcontroller and RTC

This alarm clock will have an 16x2 LCD display which will display the current time and set time. We will use few push buttons to set the alarm time whenever required. The current time will be kept in track using the **DS3231 RTC module** and we will use IIC communication to get these values from the RTC module. We have already learnt about the RTC module and how to interface it with PIC.

Materials Required:

- PIC16F877A
- 5V power source - Supply module
- 8 MHz crystal
- 33pf capacitor – 2Nos
- DS3231 RTC module
- 16*2 LCD display module
- 10K POT
- 10k and 1K resistor
- Push buttons – 5Nos
- Buzzer
- Connecting wires

The five push buttons will act as an input for setting the alarm for the required time. So one end of all the push buttons are connected to ground and the other ends are connected to PORTB pin, internal pull-up resistor will be used on these pins to avoid the pins floating. The Buzzer will act as an output and will give us a beep when the alarm gets triggered and is connected to the PORT S pin. The current time is always kept in track by the **DS3231 RTC module** from which the PIC receives the data through I2C bus, so the SCL and SDA pins of the RTC module is connected to SCL and SDA pin of the PIC controller. A LCD display is attached to the PORTD of the PIC which is used to display the current time and set time. Learn more about using DS3231 RTC module with PIC here.

CHAPTER 02 : LITERATURE SURVEY

1.Microcontroller-Based Alarm Clocks: DIY Enthusiasts:

Several online projects showcase various designs using PIC microcontrollers and different RTC modules. These offer customization and learning opportunities, but require technical expertise and breadboard setups. Commercially Available Kits: Ready-made kits provide user-friendly introductions to building microcontroller based alarm clocks. However, they often have limited features and lack the flexibility of designing from scratch.

2. Key Technologies in Existing Solutions: Real-Time Clocks (RTCs):

Essential for maintaining accurate time even during power outages. Displays: LCD displays are popular for displaying time and alarm settings, with options like 16x2 and 2x16 offering varying information visibility. Buzzers and LEDs: Audible and visual cues for alarm activation, with piezo buzzers and LEDs being common choices. Input Devices: Buttons or keypads allow users to set time,alarm parameters, and adjust settings.

3.Limitations and Areas for Improvement:

Technical Complexity: Existing microcontroller-based designs can be daunting for beginners with limited electronics knowledge. Feature

Sets: Existing kits offer basic functionalities, while DIY projects might lack sophisticated features like radio integration or smartphone connectivity.

Energy Efficiency: Microcontroller-based clocks usually require external power supplies, increasing their energy consumption. User Interface: Some designs have cumbersome button menus or lack intuitive time/alarm setting interfaces.

4. Your Project's Potential Contribution: Utilizing a PIC microcontroller and RTC, has the potential to address some of these limitations and contribute to the field of alarm clock technology.

By focusing on: Ease of Use: Simplifying the design and user interface for broader accessibility. Advanced Features: Incorporating functionalities like smartphone connectivity, customizable sound options, or light-sensitive brightness adjustments. Energy Efficiency: Utilizing low-power components and sleep modes for extended battery life.

CHAPTER 03 : METHODOLOGY

First of all we decided to make alarm clock setting embedded project as our mini project, because there were lot of benefits with this project indeed.

Creating an alarm clock setting as part of an embedded project offers several benefits, especially when compared to traditional alarm clocks. Here are some advantages:

- Customization: Embedded projects allow for a high level of customization. Users can tailor the alarm clock settings to their specific preferences, including the alarm sound, volume, snooze duration, and more.
- Integration with Other Systems: Embedded projects can be designed to integrate with other smart home or automation systems. For example, an embedded alarm clock could be linked to smart lighting or thermostat systems to create a seamless wake-up experience.
- Advanced Features: Embedded projects can incorporate advanced features such as multiple alarms, customizable sleep timers, gradual volume increase, and different alarm tones for various events.
- Energy Efficiency: Designing an embedded alarm clock allows for optimization of power consumption. Energy-efficient components and low-power modes can be implemented to ensure minimal impact on the environment and longer battery life if applicable.
- User Interface Options: With embedded projects, you can choose or design a user interface that suits the application. This could include physical buttons, touchscreens, or even voice commands, providing a more intuitive and user-friendly experience.
- Smart Connectivity: Embedded alarm clocks can have connectivity features such as Wi-Fi or Bluetooth, enabling synchronization with other devices, cloud services, or mobile applications. This connectivity can facilitate remote control and synchronization of settings.
- Software Updates and Enhancements: An embedded project allows for software updates and enhancements over time. This flexibility ensures that users can benefit from new features, bug fixes, and improvements to the alarm clock functionality.
- Learning Algorithms: Advanced embedded projects can incorporate machine learning or adaptive algorithms to analyze user behavior and optimize the alarm experienceover time. For example, the system could learn the user's preferred wake-up time based on patterns and adjust alarms accordingly.

- Compact Design: Embedded projects often enable a more compact and integrated design, making the alarm clock portable or seamlessly fitting into various environments.
- Open Source Community: If the embedded project is developed using open-source components, it can benefit from contributions and improvements from a wider community of developers and enthusiasts.
- Hardware Setup:

  Connect the PIC16F877A microcontroller to the required peripherals,such as a real time clock (RTC) module,display (LCD or LED) buttons for user input, and a speaker or buzzer for the alarm.Ensure proper power supply and ground connections.
- Real-Time Clock (RTC) Integration:

  Interface an RTC module with the microcontroller to keep track of the current time. Common RTC modules include the DS1307 or DS3231.Utilize the microcontroller's I2C or SPI communication protocol to interface with the RTC.
- User Interface:

  Implement a user interface for setting the alarm time. This could involve using buttons, a keypad, or a touchscreen display.Design a menu system or user prompts to allow users to set the alarm time, enable/disable alarms, and configure other settings.
- Alarm Logic:

  Implement the logic for checking the current time against the set alarm time. Use the RTC module to get the current time.When the alarm time is reached, trigger the alarm by activating the buzzer or speaker.
- Interrupts:

  Use interrupts to efficiently handle timekeeping and respond to user input or alarm events promptly.Set up periodic interrupts to update the time from the RTC and check for alarm conditions.
- Display:

  If using a display, update it to show the current time, alarm status, and any other relevant information.Design an easy-to-understand interface for users to interact with the alarm settings.
- Configuration Memory:

  Consider using non-volatile memory to store alarm settings and the current time, ensuring that the alarm settings persist even if power is temporarily lost.

- Testing and Debugging:

  Test the alarm clock system thoroughly to ensure accurate timekeeping, proper alarm triggering, and correct user interface functionality.Use debugging tools and techniques to identify and address any issues.
- Power Management:

  Implement power-saving features to optimize the microcontroller's energy consumption, especially if the system is battery-powered.
- Documentation:

  Document the code, circuit diagrams, and the overall design to facilitate future maintenance and development.When working with the PIC16F877A microcontroller, make sure to refer to the datasheet and reference manuals provided by Microchip Technology for detailed information on the microcontroller's features and programming. Additionally, use an integrated development environment (IDE) such as MPLAB X IDE along with a suitable compiler (e.g., XC8) for programming the microcontroller.

In second, we are searching previous project done in last by using internet and youtube for getting idea about this project, then we found several kinds of project done in several type indeed. By following that method and get lot of knowledge with this process as well as I and my group member read lot of data sheet for doing this thing better, Also, gain new idea with the seniors and asked their failures via doing projects like that, then improve our ideas and refresh with gaining new information. Finally, we decided to do our Alarm Clock Setting, First of we choosed 16f877A Microcontroller and create a code for that and finalize that in MP Lab Software, It built successfully, After doing thease things simulate our project with proteus software by using relavante components. we did that one properlly. I will upload image,(figure1,figure2, figure3) how we were doing practically in this project, So we entered for testing thease one in real world,

get breadboards, jump wires, putted code by using pic kit 3 software with pic microcontroller and other needed components. We Faced lot of challenges when were doing this, We will mention thats in discussion part. After testing this in breadboard, we moved to dote board finalizing our circuite and bouthe till finish.

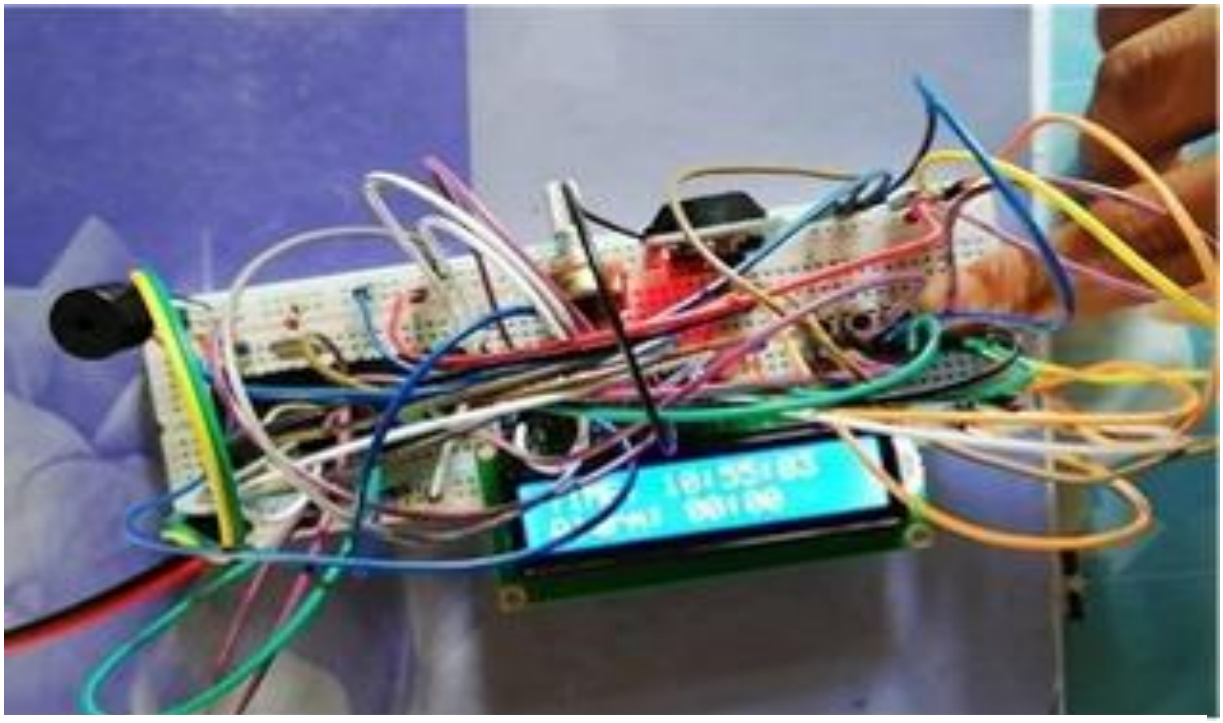

Figure 1

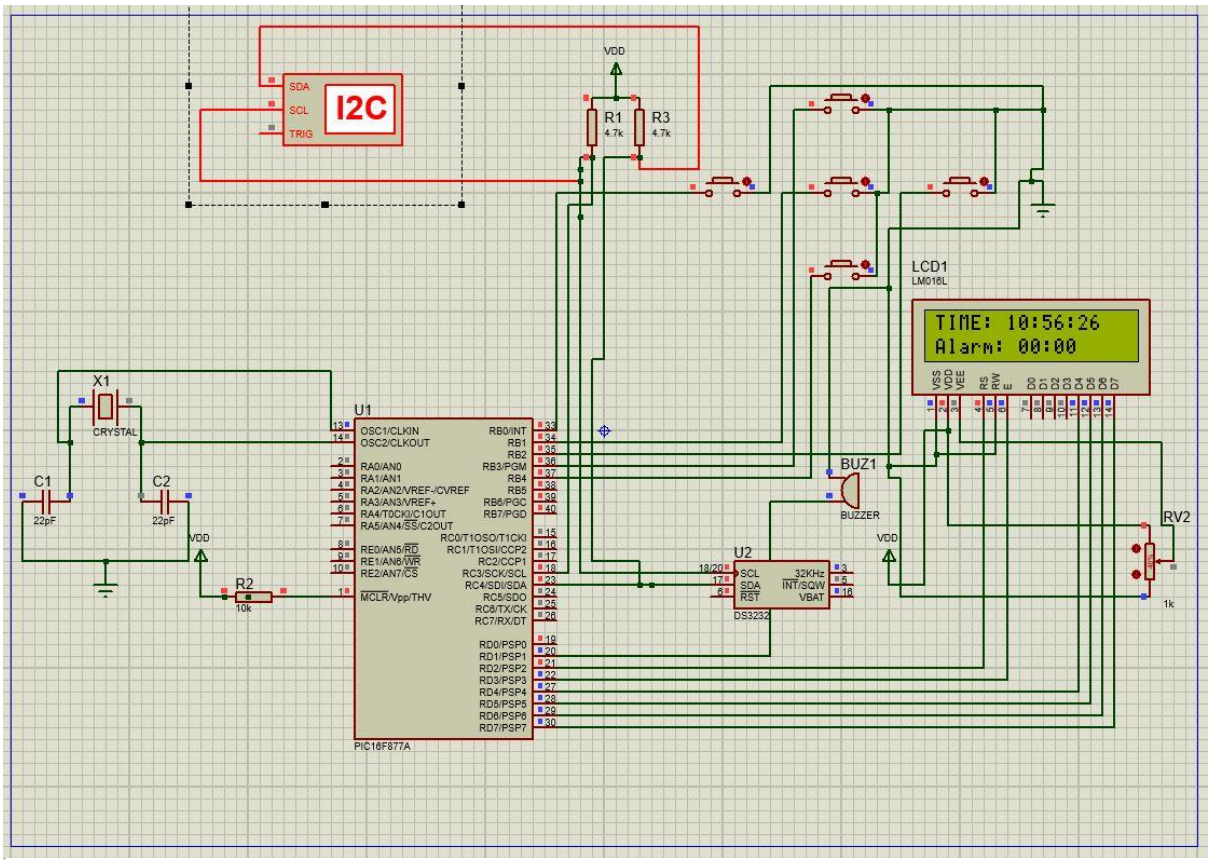

Figure2

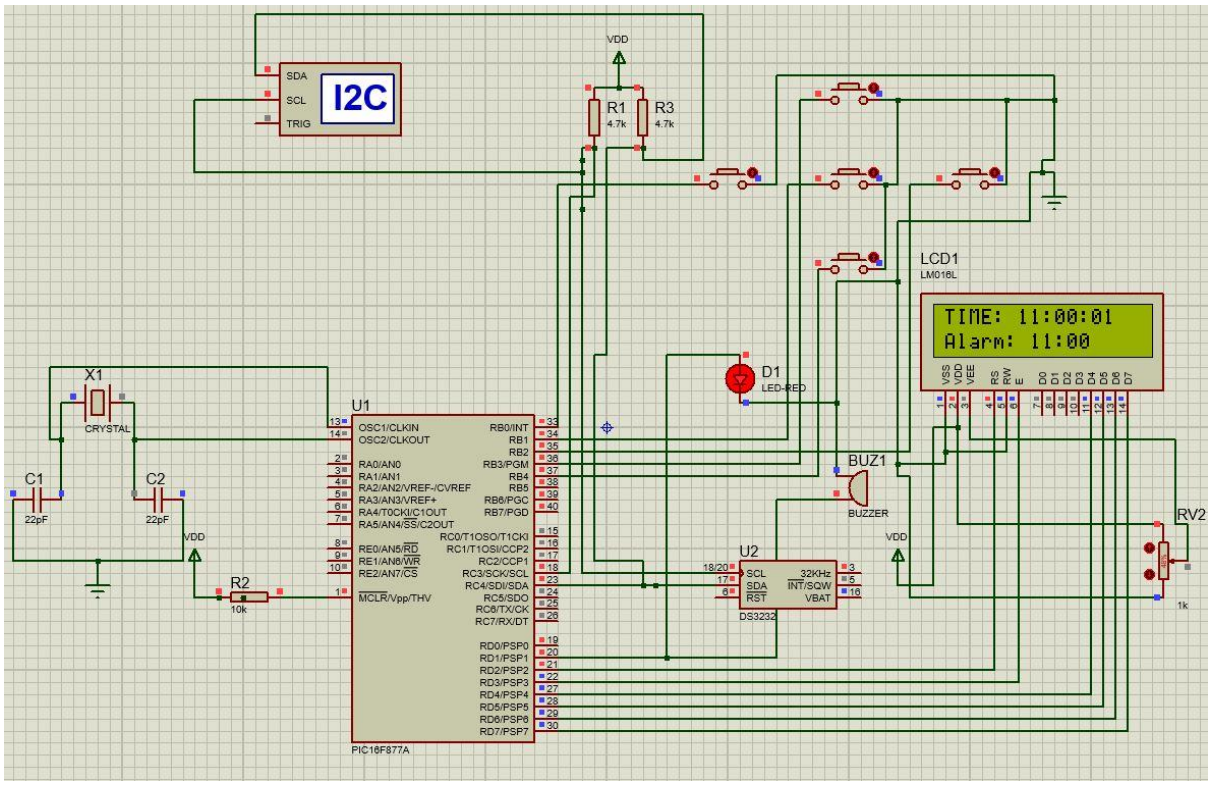
I2C
SDA
SCL
TRIG
VDD
R1
4.7k
R3
4.7k
LCD1
LM016L
TIME: 11:00:01
Alarm: 11:00
D1
LED-RED
X1
CRYSTAL
C1
22pF
C2
22pF
U1
PIC16F877A
R2
10k
BUZ1
BUZZER
U2
DS3232
RV2
1k

Figure3

CHAPTER 04 : RESULTS AND DISCUSSION

We did prepare our project using breadboard. We identified some problems have it. Some of them are, Breadboard had some errors, can't handle old push buttons. Finally we solved our problems and breadboard simulations correctly worked. Then we moved dot board simulations.

We will add images(figure4) regarding this project below, When the alarm time matches with the current time the buzzer will start beeping to alarm the user. The project has a plethora of options to build upon. The RTC module can keep track of any time and date, so you can perform a scheduled task at any time/date required. As well as, simulation in above, is represent our target project. Display the time when a task should be done or the time it should be completed on the clock and announce that time online with a message.

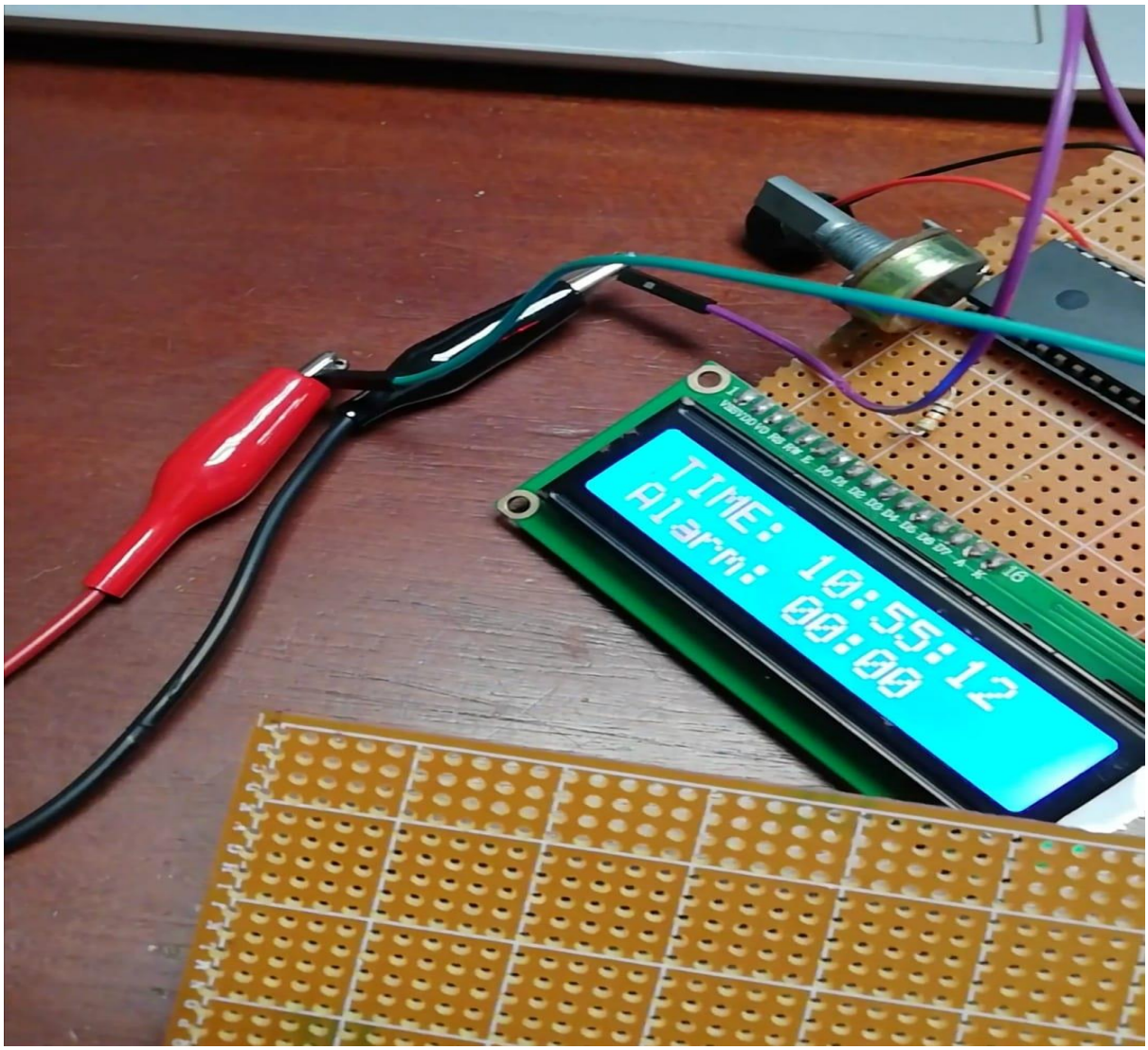

figure4

CHAPTER 05: CONCLUSIONAND FUTURE WORK

The five push buttons will act as an input for setting the alarm for the required time. So one end of all the push buttons are connected to ground and the other ends are connected to PORTB pin, internal pull-up resistor will be used on these pins to avoid the pins floating. The Buzzer will act as an output and will give us a beep when the alarm gets triggered and is connected to the PORT S pin. The current time is always kept in track by the DS3231 RTC module from which the PIC receives the data through I2C bus, so the SCL and SDA pins of the RTC module is connected to SCL and SDA pin of the PIC controller. A LCD display is attached to the PORTD of the PIC which is used to display the current time and set time You can set the alarm according to our existing system. When the buzzer is on, you can turn it off. Further, when we upgrade the system, we intend to connect it to a phone and switch it on/off. We intended to create it to determine whether they had done the work we needed to see whether we were not in that position at that moment or not.

I need to mention what are the challengers we faced, among the doing thease projects one by one and what are the solutions we get for correcting that.we were planning to display first, real word time and alarm clock time in front face display, second by pushing 4 buttons and can set the alarm using this things but, in our previous project after pushing one button going to display alarm time and real world time.Thats the first problem we had faced, as a solution we change the code part,adding real time coding part to initialization place after doing this one that error became ok. In seconde nenver displayed anythings in display and we changed 21 MegaHtz crystal occillator and replace 8 MegaHtz Occillator then another problem could be ok. As well as, simulation in above,is represent our target project.But we can’t find I2C Debugger component in any market.So we are get a meeting with Kasun sir.After doing thease Things we achieved our task successfully,
Hardware interfacing: Integrating various components like the microcontroller, RTC, display, buttons, and buzzer requires careful circuit design and programming.

Power management: Optimizing power consumption is crucial for extended battery life in portable alarm clocks. Software development: Programming the microcontroller to handle timekeeping, user interaction, alarm functionalities, and other features requires knowledge of embedded C programming. Large-scale projects such as factories were created to meet the needs and wants of the living population of the existing society. There, people with special

training for certain tasks were created. It was necessary to know whether a certain task was done by a certain person using a group of other people.

This is our working alarm clock. Still we can set the alarm and turn it off. Here we decided to use a Wi-Fi module to send a message to the phone after the alarm is turned off. To ensure that the higher officials have started the work and to evaluate that the employees who have to do the work have for the time.

APPENDICES – PROGRAM CODE

Alarm clock PIC.c

```c
/*
 * Program: Alarm Clock Using PIC.
 */
#pragma config FOSC = HS        // Oscillator Selection bits (HS oscillator)
#pragma config WDTE = OFF       // Watchdog Timer Enable bit (WDT disabled)
#pragma config PWRTE = ON       // Power-up Timer Enable bit (PWRT enabled)
#pragma config BOREN = ON       // Brown-out Reset Enable bit (BOR enabled)
#pragma config LVP = OFF        // Low-Voltage (Single-Supply) In-Circuit Serial Programming Enable bit (RB3 is digital I/
#pragma config CPD = OFF        // Data EEPROM Memory Code Protection bit (Data EEPROM code protection off)
#pragma config WRT = OFF        // Flash Program Memory Write Enable bits (Write protection off; all program memory may be
#pragma config CP = OFF         // Flash Program Memory Code Protection bit (Code protection off)

#define _XTAL_FREQ 8000000 //We are running on 8MHz crystal


//Define the LCD pins
#define RS RD2 //Reset pin of LCD
#define EN RD3 //Enable pin of LCD
#define D4 RD4 //Data bit 0 of LCD
#define D5 RD5 //Data bit 1 of LCD
#define D6 RD6 //Data bit 2 of LCD
#define D7 RD7 //Data bit 3 of LCD

//Define Buttons
#define MB RB1 //The middle button
#define LB RB0 //Left button
#define RB RB2 //Right button
#define UB RB3 //Upper Button
#define BB RB4 //Bottom button

//Define Buzz
#define BUZZ RD1 //Buzzer is connected to RD1
```

```c
#define BUZZ RD1 //Buzzer is connected to RD1

/*Set the current value of date and time below*/
int sec = 00;
int min = 55;
int hour = 10;
int date = 06;
int month = 05;
int year = 18;
/*Time and Date Set*/

/*Vars for Alarm clock*/
char set_alarm = 0;
char trigger_alarm = 0;
char pos = 8;
char jump = 0;

char alarm_h0 = 0;
char alarm_h1 = 0;
char alarm_m0 = 0;
char alarm_m1 = 0;
int alarm_val[4] = {0, 0, 0, 0};
/*End of var declaration*/

#include <xc.h>
#include "lcd.h" //Header for using LCD module
#include "PIC16F877a_I2C.h" // Header for using I2C protocal
#include "PIC16F877a_DS3231.h" //Header for using DS3231 RTC module

int main() {
```

```c
int main() {

    TRISD = 0x00; //Make Port D pins as output for LCD interfacing
    TRISB = 0xFF; //Switchs are declared as input pins
    OPTION_REG = 0b00000000; //Enable pull up Resistor on port B for switchs
    BUZZ = 0; //Turn of buzzer

    Lcd_Start(); // Initialize LCD module

    I2C_Initialize(100); //Initialize I2C Master with 100KHz clock

    //Remove the below line once time and date is set for the first time.
    Set_Time_Date(); //set time and date on the RTC module

    //Give an intro message on the LCD
        Lcd_Clear();
        Lcd_Set_Cursor(1,1);
        Lcd_Print_String("Alarm Clock");
        Lcd_Set_Cursor(2,1);
        Lcd_Print_String("SLTC");
        __delay_ms(1500);

    while (1) {

        Update_Current_Date_Time(); //Read the current date and time from RTC module

        //Split the into char to display on lcd
        char sec_0 = sec % 10;
        char sec_1 = (sec / 10);
        char min_0 = min % 10;
        char min_1 = min / 10;
        char hour_0 = hour % 10;
```

```c
        char hour_1 = hour / 10;


        //Display the Current Time on the LCD screen
        Lcd_Clear();
        Lcd_Set_Cursor(1, 1);
        Lcd_Print_String("TIME: ");
        Lcd_Print_Char(hour_1 + '0');
        Lcd_Print_Char(hour_0 + '0');
        Lcd_Print_Char(':');
        Lcd_Print_Char(min_1 + '0');
        Lcd_Print_Char(min_0 + '0');
        Lcd_Print_Char(':');
        Lcd_Print_Char(sec_1 + '0');
        Lcd_Print_Char(sec_0 + '0');



        //Display the Date on the LCD screen
        Lcd_Set_Cursor(2, 1);
        Lcd_Print_String("Alarm: ");
        Lcd_Print_Char(alarm_val[0] + '0');
        Lcd_Print_Char(alarm_val[1] + '0');
        Lcd_Print_Char(':');
        Lcd_Print_Char(alarm_val[2] + '0');
        Lcd_Print_Char(alarm_val[3] + '0');

        __delay_ms(50);


        //Use middle button to check if alarm has to be set
        if (MB == 0 && set_alarm == 0) { //If middle button is pressed and alarm is not turned on
```

```
            while (!MB); //Wait till button is released
            set_alarm = 1; //start setting alarm value
        }
        if (MB == 0 && set_alarm == 1) { //If middle button is pressed and alarm is not turned off
            while (!MB); //Wait till button is released
            set_alarm = 0; //stop setting alarm value
        }


        //If alarm has to be navigate through each digit
        if (set_alarm == 1) {
            if (LB == 0) { //If left button is pressed
                while (!LB); //Wait till button is released
                pos--; //Then move the cursor to left
            }
            if (RB == 0) { //If right button is pressed
                while (!RB); //Wait till button is released
                pos++; //Move cursor to right
            }

            if (pos >= 10) //eliminate ":" symbol if cursor reaches there
            {
                jump = 1; //Jump over the ":" symbol
            } else
                jump = 0; //get back to normal movement


            if (UB == 0) { //If upper button is pressed
                while (!UB); //Wait till button is released
                alarm_val[(pos - 8)]++; //Increase that particular char value
            }
            if (BB == 0) { //If lower button is pressed
```

```
            if (BB == 0) { //If lower button is pressed
                while (!UB); //Wait till button is released
                alarm_val[(pos - 8)]--; //Decrease that particular char value
            }

            Lcd_Set_Cursor(2, pos + jump);
            Lcd_Print_Char(95); //Display "_" to indicate cursor position
        }

        //IF alarm is set Check if the set value is equal to current value
        if (set_alarm == 0 && alarm_val[0] == hour_1 && alarm_val[1] == hour_0 && alarm_val[2] == min_1 && alarm_val[3] ==
            trigger_alarm = 1; //Turn on trigger if value match


        if (trigger_alarm) { //If alarm is triggered
            //Beep the buzzer
            BUZZ = 1;
            __delay_ms(500);
            BUZZ = 0;
            __delay_ms(500);
        }

        __delay_ms(200);//Update interval

    }

    return 0;
}
```

References

[1] “PIC16F877A Microcontroller Based Digital Alarm Clock,” circuitdi gest.com. https://circuitdigest.com/microcontroller-projects/pic16f877a based-digital-alarm-clock

[2] “What is a Real Time Clock (RTC)?,” ECS Inc. https://ecsxtal.com/what is-a-real-time-clock-rtc/: :text=A

[3] “WiFi Module - ERC Handbook,” erc-bpgc.github.io. https://erc bpgc.github.io/handbook/electronics/Modules/wifimodule/˚

[4] “Alarm Clock using PIC Microcontroller and RTC Module,” www.youtube.com. https://www.youtube.com/watch?v=NPZdU x4FWM (accessed Jan. 17, 2024).

[5] “CCS :: View topic - alarm clock from 16f877a,” www.ccsinfo.com. https://www.ccsinfo.com/forum/viewtopic.php?t=48099 (accessed Jan. 18, 2024)